

AutoIntern

Roadmap, Work Division & Task Execution Guide

MVP Build · Lean Team Structure · Technical How-To Blueprints

§1	8-Week Master Roadmap Chronological timeline from setup to launch
§2	Work Division Role A, B, C lean team ownership matrix
§3	Task 1 – Supabase & Auth JWT validation, FastAPI security dependency
§4	Task 2 – Resume Extraction PDF parsing + Claude-3-Haiku JSON structuring
§5	Task 3 – Match Scoring Haiku-based 0-100 skill-to-JD scoring
§6	Task 4 – Resume Tailoring Sonnet-powered bullet rewriting w/ factual guard
§7	Task 5 – Web Scraping BeautifulSoup scrapers + DB caching engine
§8	Task 6 – Tracking Pixel 1x1 PNG endpoint + inbound reply webhooks
§9	Task 7 – Split-Screen UI React Vite Copilot with clipboard APIs

1 8-WEEK MASTER ROADMAP

This chronological timeline is optimised to deliver a functional MVP into students' hands by Week 4, followed by polishing, monetisation, and full launch scaling in Weeks 5–8.

W1

Found.

W2

AI Core

W3

Scrapers

W4

Email

W5

Scoring

W6

Kanban

W7

Copilot

W8

Launch

W1

Foundations & Scaffolding

- Set up FastAPI monorepo directory layout.
- Configure Supabase project and execute database DDL scripts.
- Scaffold the React Vite frontend and build the vanilla CSS global styling tokens.

W2

AI Core & Parsing Pipeline

- Integrate the Anthropic Claude API SDK on the backend.
- Build the PDF text extractor and the AI resume JSON parser.
- Verify structured JSON extraction and database writes.

W3

Job Aggregation & Caching Engine

- Write RSS feed parsers and local HTML scraper scripts.
- Integrate a 3-second rate-limiting utility to respect target sites.
- Set up background schedulers to scrape and cache listings in PostgreSQL every 12 hours.

W4

The Outgoing Email Channel — MVP Milestone

- Integrate the Resend API or custom SMTP mailer.
- Build the basic single-page application dashboard showing available jobs.
- Test sending a basic tailored email with a custom PDF attachment.

W5

Match Scoring & Feed Ranking

- Build the AI Match Scoring endpoint (Haiku-based 0-100 score).
- Connect the React frontend to fetch jobs sorted dynamically by Match Score.
- Build search, filtering, and role-based categorisation on the dashboard.

W6

The Kanban Board & Tracking Pixel

- Build the interactive frontend Kanban Board UI (Draft -> Applied -> Seen).
- Implement the `/track/open/{app_id}.png` tracking pixel endpoint in FastAPI.
- Configure the Inbound Reply Webhook to catch recruiter emails.

W7

Split-Screen Copilot & Analytics

- Build the split-screen double-pane editor UI for portal-based applications.
- Implement pre-generated Q&A; answer-blocks and one-click PDF downloading.
- Create a simple Weekly Analytics dashboard for users (open rates, applications sent).

W8

Razorpay Integration & Campus GTM Launch

- Integrate Razorpay/Stripe checkout hooks to enforce the 10-application freemium limit.
- Build referral link code systems (refer a friend to get 10 extra monthly credits).
- Launch the Campus Ambassador WhatsApp/LinkedIn viral campaigns.

2 WORK DIVISION — LEAN TEAM STRUCTURE

Tasks are divided among three specialised roles to execute the roadmap efficiently. Each role owns a distinct layer of the stack, enabling parallel development with minimal blockers.

Role A

Full Stack Lead

(Backend & AI Architect)

- FastAPI API routing & middleware
- Supabase DB sessions & Auth JWT
- Claude API integration
- Email delivery routes (Resend/SES)

Primary Weeks: 1, 2, 4, 6, 8

Role B

Frontend Developer

(UI/UX & Interactions)

- React UI rendering & state management
- Vanilla CSS design tokens (glassmorphism)
- Kanban board & Split-screen Copilot view
- Razorpay client integrations

Primary Weeks: 1, 4, 5, 6, 7, 8

Role C

Scraper / Data Engineer

(Pipelines & Background Jobs)

- Scrapy/BeautifulSoup scrapers
- HTML structure parsers
- Background schedulers (Celery/Cron)
- Anti-bot bypass mechanisms

Primary Weeks: 3, 5, 7

3 TASK 1: SETTING UP SUPABASE DATABASE & AUTH JWT VALIDATION

BACKEND · AUTH

Goal: Protect API routes so only students logged in via Supabase can query the backend.

How to Accomplish

1

Create Project

Create a free project on Supabase. Navigate to the SQL Editor.

2

Execute DDL

Execute your DDL SQL schema in the SQL Editor to initialise the tables (*users*, *resumes*, etc.).

3

Security Dependency

In *backend/app/core/security.py*, create a FastAPI dependency that extracts and validates the Supabase bearer JWT token from incoming request headers using PyJWT or HTTPBearer.

Implementation File: *backend/app/core/security.py*

PYTHON

```
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from jose import jwt
import os

security = HTTPBearer()
SUPABASE_JWT_SECRET = os.environ.get('SUPABASE_JWT_SECRET')

async def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security)
):
    token = credentials.credentials
    try:
        payload = jwt.decode(
            token, SUPABASE_JWT_SECRET,
            algorithms=['HS256'], audience='authenticated'
        )
        user_id = payload.get('sub')
        return {'id': user_id, 'email': payload.get('email')}
    except Exception:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail='Invalid or expired credentials',
        )
```

4 TASK 2: RESUME EXTRACTION AND AI STRUCTURING

AI · PARSING

Goal: Convert raw resume text into structured JSON skills and experience using Claude-3-Haiku.

How to Accomplish

1

Extract Text

Use a library like *pypdf* or *pdfplumber* to extract raw string text from uploaded resume files.

2

Craft Prompt

Write an AI prompt targeting *claude-3-haiku* that forces JSON-only output with fields: name, email, skills, experience, education.

3

Parse Response

Call the Anthropic API, receive the structured text, and parse it with *json.loads()* into a Python dict for database insertion.

Implementation File: `backend/app/services/ai_service.py`

PYTHON

```
import json
from anthropic import Anthropic

client = Anthropic()

def extract_structured_resume(raw_pdf_text: str) -> dict:
    prompt = f'''
    Analyze this raw text from a student's resume and parse it
    into a clean JSON structure. Return ONLY valid JSON.
    JSON Fields:
    - name: string
    - email: string
    - skills: list of strings
    - experience: list of dicts (company, role, bullets)
    - education: list of strings

    Raw Resume Text: {raw_pdf_text}
    '''
    response = client.messages.create(
        model='claude-3-haiku-20240307',
        max_tokens=1500,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return json.loads(response.content[0].text)
```

5 TASK 3: JOB MATCH SCORING

AI · SCORING

Goal: Compare a student's skills with a Job Description (JD) and return a Match Score from 0 to 100.

How to Accomplish

1

Query Haiku

Use *Claude 3 Haiku* for cheap bulk scoring — it's fast and cost-efficient for high-volume ranking operations.

2

Structured Prompt

Pass the list of user skills and the job description. Instruct the model to return only JSON with a *score* (int) and a *reason* (string).

3

Parse & Store

Parse the JSON response and store the score alongside the job listing in PostgreSQL for real-time feed sorting.

Implementation File: `backend/app/services/ai_service.py`

PYTHON

```
def calculate_match_score(user_skills: list, job_description: str) -> dict:
    prompt = f'''
    Analyze the student's skills against the job requirements.
    Return ONLY a JSON response containing:
    1. 'score': integer from 0 to 100
    2. 'reason': a single clear sentence explaining the fit.

    Student Skills: {' '.join(user_skills)}
    Job Description: {job_description}
    '''

    response = client.messages.create(
        model='claude-3-haiku-20240307',
        max_tokens=300,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return json.loads(response.content[0].text)
```

6 TASK 4: PREMIUM RESUME TAILORING (HUMAN-IN-THE-LOOP GUARD)

AI · TAILORING

Goal: Rewrite resume bullet points targeting a specific job without hallucinating or fabricating facts.

How to Accomplish

1

Use Sonnet

Query the premium *claude-3-5-sonnet* model for higher-quality rewrites — this is a paid feature.

2

Factual Guard

CRITICAL RULE: Force the model to retain all historical facts (company names, dates, core tasks) while adapting action verbs and keyword density.

3

Return Tailored JSON

Return a JSON object matching the original resume format but with optimised bullet points and a newly drafted *cover_letter* field.

Implementation File: `backend/app/services/ai_service.py`

PYTHON

```
def tailor_resume_bullets(resume_json: dict, job_description: str) -> dict:
    prompt = f'''
    You are an expert resume optimizer.
    Reword the bullet points to target keywords in the Job Description.
    CRITICAL RULE: Keep all facts (company names, dates, core tasks)
    100% true. Do NOT invent new credentials.

    Job Description: {job_description}
    Student Profile: {json.dumps(resume_json)}

    Return ONLY a JSON object matching the original format
    but with optimized bullets and a 'cover_letter' field.
    '''

    response = client.messages.create(
        model='claude-3-5-sonnet-20240620',
        max_tokens=2000,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return json.loads(response.content[0].text)
```

7 TASK 5: WEB SCRAPING AND DB CACHING

SCRAPING · DATA

Goal: Crawl internship listings from career boards and cache them in PostgreSQL to avoid API blocks.

How to Accomplish

1

BeautifulSoup Scraper

Use *BeautifulSoup* to parse the HTML of career listing pages and extract job card elements.

2

Rate Limiting

Throttle calls with a **3-second sleep timer** between requests to mimic natural user behaviour and avoid bot detection.

3

Unique URL Constraint

Store crawled results in PostgreSQL with a unique URL constraint to prevent duplicate listings across scraper runs.

Implementation File: `backend/app/services/scraper_service.py`

PYTHON

```
import urllib.request
from bs4 import BeautifulSoup
import time

def scrape_internships():
    url = 'https://example-careers-board.com/internships'
    req = urllib.request.Request(
        url, headers={'User-Agent': 'Mozilla/5.0'}
    )
    with urllib.request.urlopen(req) as response:
        html = response.read()
    soup = BeautifulSoup(html, 'html.parser')
    listings = soup.find_all('div', class_='job-card')
    jobs = []
    for card in listings:
        title = card.find('h2').text.strip()
        company = card.find('div', class_='company').text.strip()
        jd_text = card.find('p', class_='description').text.strip()
        source_url = card.find('a')['href']
        jobs.append({'title': title, 'company': company,
                    'jd_text': jd_text, 'source_url': source_url})
        time.sleep(3.0) # Throttling - respect target site
    return jobs
```


8 TASK 6: 1x1 TRACKING PIXEL & INBOUND WEBHOOKS

TRACKING · WEBHOOKS

Goal: Track when outbound emails are opened by HR recruiters and when they send reply emails.

How to Accomplish

1

Inject Pixel Tag

Embed a unique `<img>` tag in the HTML email body, mapping to the application's `app_id` on the tracking endpoint.

2

FastAPI Pixel Route

Build a FastAPI GET route at `/track/open/{app_id}.png` that returns a raw transparent 1x1 PNG and updates the application status to `seen`.

3

Cache-Control Headers

Set no-cache headers on the pixel response so the recruiter's email client re-fetches it each open rather than serving from browser cache.

Implementation File: `backend/app/api/applications.py`

PYTHON

```
from fastapi import APIRouter, Response, Depends
from sqlalchemy.orm import Session
from app.core.database import get_db
from app.models.schema import Application

router = APIRouter()

# 1x1 transparent PNG bytes
TRANSPARENT_PNG = b'\x89PNG\r\n\x1a\n\x00\x00\x00\rIHDR'
TRANSPARENT_PNG += b'\x00\x00\x00\x01\x00\x00\x00\x01\x08\x06'

@router.get('/track/open/{app_id}.png')
def track_email_open(app_id: str, db: Session = Depends(get_db)):
    app = db.query(Application).filter(
        Application.id == app_id).first()
    if app and app.status == 'applied':
        app.status = 'seen'
        db.commit()
    return Response(
        content=TRANSPARENT_PNG,
        media_type='image/png',
        headers={
            'Cache-Control': 'no-cache, no-store, must-revalidate',
            'Pragma': 'no-cache', 'Expires': '0'
        }
    )
```

9 TASK 7: REACT VITE SPLIT-SCREEN COPILOT

FRONTEND · REACT

Goal: Provide a side-by-side interface to help students copy tailored answers and upload resumes to portals in seconds.

How to Accomplish

1

Split Layout

Render a split-screen dashboard — **Left pane** embeds the external portal in an `<iframe>`; **Right pane** shows the AutoIntern Copilot sidebar.

2

Clipboard API

Use the standard `navigator.clipboard.writeText()` API for one-click copy buttons on cover letters and custom Q&A; answers.

3

Mark as Applied

After the student submits on the portal, clicking "**Confirm Application Submitted**" fires a PUT request to update the application status to *applied*.

Implementation File: [frontend/src/pages/PortalCopilot.tsx](#)

```
import React, { useState } from 'react';

export const PortalCopilot = ({ jobUrl, tailoredResumeUrl,
                                coverLetterText }) => {
  const [copied, setCopied] = useState(null);

  const copyToClipboard = (text, label) => {
    navigator.clipboard.writeText(text);
    setCopied(label);
    setTimeout(() => setCopied(null), 2000);
  };

  return (
    <div className='flex h-screen w-full bg-slate-950'>
      <div className='w-1/2 border-r border-slate-800'>
        <iframe src={jobUrl} className='w-full h-full bg-white' />
      </div>
      <div className='w-1/2 p-6 flex flex-col overflow-y-auto'>
        <h2 className='text-xl font-bold text-indigo-400 mb-4'>
          AutoIntern Copilot
        </h2>
        <a href={tailoredResumeUrl} download
          className='px-4 py-2 bg-indigo-600 rounded-lg text-sm'>
          Download Tailored PDF
        </a>
        <button onClick={() => copyToClipboard(coverLetterText, 'cover')}
          className='mt-4 text-xs px-2 py-1 bg-slate-800 rounded'>
          {copied === 'cover' ? 'Copied!' : 'Copy Cover Letter'}
        </button>
        <button className='mt-auto py-3 bg-indigo-500 rounded-xl font-semibold'>
          Confirm Application Submitted
        </button>
      </div>
    </div>
  );
};
```

Full Stack Summary

Component	Technology / Tool	Week
Database & Auth	Supabase PostgreSQL + JWT	W1-W2
Resume Parsing	Claude-3-Haiku + pypdf	W2
Job Scraping	BeautifulSoup + Celery/Cron	W3
Email Delivery	Resend API / AWS SES	W4

Match Scoring	Claude-3-Haiku (0–100 score)	W5
Tracking Pixel	FastAPI PNG endpoint	W6
Split-Screen Copilot	React Vite + Clipboard API	W7
Monetisation & Launch	Razorpay + Referral Links	W8